

Sapna Sarees Platform

Software Requirement Document (SRD) & Google Cloud Infrastructure Specifications

1. Purpose & Scope

This document defines the software requirements, target cloud infrastructure, and operational parameters for the "Sapna Sarees" e-commerce platform. The system is engineered to support a user base of 11,000 customers and a catalog of 1,000 saree products across three synchronized channels: a customer web storefront, WhatsApp (automated catalog and chat), and Instagram (Direct Messaging). All transactions, inventory changes, and customer accounts synchronize through a central backend and unified database.

2. Technology Stack & Programming Languages

- **Frontend Framework:** React (with Vite build tooling) utilizing modern JavaScript/TypeScript and Tailwind CSS for the customer boutique and administrative dashboard.
- **Backend Language & Runtime:** Node.js (v20+ LTS) with the Express.js framework for REST API routing, authentication, and webhook handling.
- **Database & ORM:** PostgreSQL (v15+) utilizing Prisma ORM for schema definition, type-safe migrations, and relationship management.
- **Automation Orchestration:** n8n Automation Engine deployed as a containerized Node.js service for Meta Cloud Webhooks.

3. Google Cloud Platform (GCP) Hosting Architecture & Services

Enterprise GCP Service Mapping

The platform leverages dedicated Google Cloud services to isolate frontends, API processing, automation workloads, and data persistence with serverless elasticity and high security.

- **Firebase Hosting (Frontend Hosting & Global CDN):**
 - **Customer Storefront:** Hosts `www.sapnasarees.com` with automated CI/CD directly from the GitHub repository, caching static assets globally via Google's edge network for lightning-fast catalog browsing.
 - **Atelier Staff Dashboard:** Hosts the administrative portal on a dedicated subdomain `atelier.sapnasarees.com` with strict RBAC security headers.
- **Cloud Run (Containerized Serverless API Gateway):**
 - Runs the Node.js/Express backend inside a managed container environment that automatically scales compute up during viral traffic spikes and scales down during low-traffic periods, optimizing operational costs.
- **Google Compute Engine (Automation Workloads):**
 - Dedicated Virtual Machine instance (`e2-standard-2`) hosting the n8n container, ensuring 24/7 continuous listening for WhatsApp and Instagram webhook events without impacting storefront performance.
- **Cloud SQL for PostgreSQL (Managed Database Persistence):**
 - Fully managed PostgreSQL instance located in the primary GCP region, configured with automated daily backups, point-in-time recovery, SSL encryption at rest and in transit, and private VPC peering.

4. Detailed Hardware & Infrastructure Allocations

GCP Service / Role	Configuration / Machine Type	vCPU	RAM	Storage	Target Purpose
Firestore Hosting	Serverless Global CDN	Managed	Managed	Edge Cache	Instant global loading of Customer Storefront & Atelier Dashboard.
Cloud Run (API)	Containerized Microservice	Up to 4 vCPUs	8 GB (Autoscaling)	Ephemeral Container	Handles React HTTP requests, JWT verification, and order processing transactions.
Compute Engine (n8n)	e2-standard-2	2 vCPUs	8 GB	40 GB Balanced PD	Processes WhatsApp Business API & Instagram Graph API webhook events 24/7.
Cloud SQL (PostgreSQL)	db-custom-2-8192	2 vCPUs	8 GB	100 GB SSD	Stores unified User accounts, 1k product inventory, orders, and StockLedger.

5. Automation & Omnichannel Specifications (n8n Engine)

n8n Middleware Integration on GCP

The n8n automation engine operates as an event-driven gateway between Meta's Cloud APIs and the backend REST service hosted on Cloud Run.

- WhatsApp Cloud API Webhooks:** Receives customer messages, translates keyword intents ("Catalog", "Banarasi", "Order #"), queries `GET /api/products` on Cloud Run, and outputs Meta-compliant Multi-Product Messages (MPM).
- Instagram Graph API Webhooks:** Catches story mentions, comments, and DMs, delivering instant mini-catalog carousels and quick-checkout bridges.
- Automated Order Generation:** Dispatches `POST /api/orders` with customer identifiers, automatically decrementing inventory in Cloud SQL and updating staff Kanban boards.

6. Database Architecture & Security Requirements

- Unified User Table:** Single authentication entity containing a `Role` enum (`'CUSTOMER'` , `'EMPLOYEE'` , `'ADMIN'` , `'OWNER'`).
- Granular Employee Permission Flags:** Boolean flags on the User model (`canManageCatalog` , `canViewRevenue` , `canManageOrders`) controlling dashboard actions.
- Dual-Frontend Security Guard:** Login requests pass a `portalSource` payload; requests from `'CUSTOMER'` targeting `'dashboard'` are rejected with HTTP 403 Forbidden.

- **Stock Ledger Inventory Separation:** Product inventory counts are decoupled into a dedicated `StockLedger` table to preserve an immutable audit trail.